

Easy2.AcquireCom

Table of Contents

1 Purpose.....	3
2 Operating cycle.....	3
3 Handling errors.....	4
4 Setup.....	5
5 Components.....	5
5.1 Component types.....	5
5.2 Id, tags, and enumeration.....	5
5.3 Selecting components for measurement.....	6
6 Data transmission.....	6
6.1 Where to get the acquired data from.....	6
6.2 Transmission and timing.....	6
6.3 Data transfer operation.....	7
6.4 Reading out the remaining data after stop.....	7
6.5 Post-acquisition data retrieval.....	7
7 Redistribution.....	7
8 Interface Reference.....	8
8.1 IDeviceManager.....	8
8.1.1 Initialize (ConfigDir String).....	8
8.1.2 Setup (hWndParent Window), returns Bool.....	8
8.1.3 GetCurrentDevice (), returns IDevice.....	8
8.1.4 GetLastErrorText (), returns String.....	8
8.1.5 ClearLastErrorText ().....	9
8.2 IDevice.....	10
8.2.1 GetComponentById (Id String), returns IComponent.....	10
8.2.2 GetComponentCount (), returns Integer.....	10
8.2.3 GetComponent (Index Integer), returns IComponent.....	10
8.2.4 ClearComponentFilter ().....	10
8.2.5 AddComponentFilterTags (Tags String).....	10
8.2.6 SetComponentFilterTags (Tags String).....	10
8.2.7 GetTagsByPrefix (Prefix String), returns Array of String.....	10
8.2.8 GetTagName (Tag String), returns String.....	11
8.2.9 Activate ().....	11
8.2.10 GetSession (), returns String.....	11
8.2.11 Transfer (), returns mask.....	11
8.2.12 Record ().....	11
8.2.13 Stop ().....	12
8.2.14 Deactivate ().....	12
8.3 IComponent.....	13
8.3.1 GetId (), returns String.....	13
8.3.2 GetTags (), returns Array of String.....	13
8.4 IAnalogInput.....	14
8.4.1 Enable ().....	14
8.4.2 Disable ().....	14
8.4.3 Enabled (), returns Integer.....	14
8.4.4 GetMaxQuantCount (), returns Integer.....	14
8.4.5 SetDesiredFrequency (DesiredFrequency Float64).....	14
8.4.6 GetDesiredFrequency (), returns Float64.....	14
8.4.7 GetFrequency (), returns Float64.....	14
8.4.8 GetUnits (), returns String.....	15
8.4.9 CreateCompatibleBuffer (Buffer Variant).....	15
8.4.10 GetQuantCount (), returns Integer.....	15
8.4.11 GetQuants (SrcIndex Integer, Count Integer, Buffer Variant, DstIndex Integer).....	15
8.4.12 GetStartTimeHW (), returns Float64.....	15

8.4.13	GetStartTimeSW (), returns Float64.....	15
8.4.14	RecoveryEnable(), returns BOOL.....	16
8.4.15	RecoveryStart(String Session).....	16
8.4.16	RecoveryBlockAsk(Index Integer64, Count Integer).....	16
8.4.17	RecoveryBlockGet(Buffer Variant, DstIndex Integer).....	16
8.5	IDigitalInput.....	17
8.5.1	Enable ().....	17
8.5.2	Disable ().....	17
8.5.3	Enabled (), returns Integer.....	17
8.5.4	GetMaxQuantCount (), returns Integer.....	17
8.5.5	SetDesiredFrequency (DesiredFrequency Float64).....	17
8.5.6	GetDesiredFrequency (), returns Float64.....	17
8.5.7	GetFrequency (), returns Float64.....	17
8.5.8	CreateCompatibleBuffer (Buffer Variant).....	18
8.5.9	GetQuantCount (), returns Integer.....	18
8.5.10	GetQuants (SrcIndex Integer, Count Integer, Buffer Variant, DstIndex Integer).....	18
8.5.11	GetStartTimeHW (), returns Float64.....	18
8.5.12	GetStartTimeSW (), returns Float64.....	18
8.6	IMotionInput.....	20
8.6.1	Enable ().....	20
8.6.2	Disable ().....	20
8.6.3	Enabled (), returns Integer.....	20
8.6.4	GetMaxQuantCount (), returns Integer.....	20
8.6.5	SetDesiredFrequency (DesiredFrequency Float64).....	20
8.6.6	GetDesiredFrequency (), returns Float64.....	20
8.6.7	GetFrequency (), returns Float64.....	20
8.6.8	CreateCompatibleBuffer (Buffer Variant).....	21
8.6.9	GetQuantCount (), returns Integer.....	21
8.6.10	GetQuants (SrcIndex Integer, Count Integer, Buffer Variant, DstIndex Integer).....	21
8.6.11	GetStartTimeHW (), returns Float64.....	21
8.6.12	GetStartTimeSW (), returns Float64.....	21
8.6.13	RecoveryEnable(), returns BOOL.....	22
8.6.14	RecoveryStart(String Session).....	22
8.6.15	RecoveryBlockAsk(Index Integer64, Count Integer).....	22
8.6.16	RecoveryBlockGet(Buffer Variant, DstIndex Integer).....	22
8.7	IMatrixInput.....	23
8.7.1	Enable ().....	23
8.7.2	Disable ().....	23
8.7.3	Enabled (), returns Integer.....	23
8.7.4	GetSizeX (), returns Integer.....	23
8.7.5	GetSizeY (), returns Integer.....	23
8.7.6	GetResX (), returns Real64.....	23
8.7.7	GetResY (), returns Real64.....	23
8.7.8	GetRes (), returns Real64.....	23
8.7.9	GetMaxQuantCount (), returns Integer.....	24
8.7.10	SetDesiredFrequency (DesiredFrequency Float64).....	24
8.7.11	GetDesiredFrequency (), returns Float64.....	24
8.7.12	GetFrequency (), returns Float64.....	24
8.7.13	CreateCompatibleBuffer (Buffer Variant).....	24
8.7.14	GetQuantCount (), returns Integer.....	24
8.7.15	GetQuant (SrcIndex Integer, Buffer Variant).....	24
8.7.16	GetStartTimeHW (), returns Float64.....	25
8.7.17	GetStartTimeSW (), returns Float64.....	25
9	The list of changes.....	26

1 Purpose

Easy2.AcquireCom is a data acquisition library.

- It serves as hardware-abstraction layer, so a correctly-written application must be able to work with different devices without recompilation.
- It is able to work with several devices simultaneously, presenting them to the client application as one joined virtual device with a single operating state, and utilizing hardware synchronization between different data streams whenever possible.
- It has COM interface to client applications, so you can create applications in different programming languages. All COM objects of the library besides their native interfaces support the interface IDispatch, so you can use it in visual COM-enabled environments.

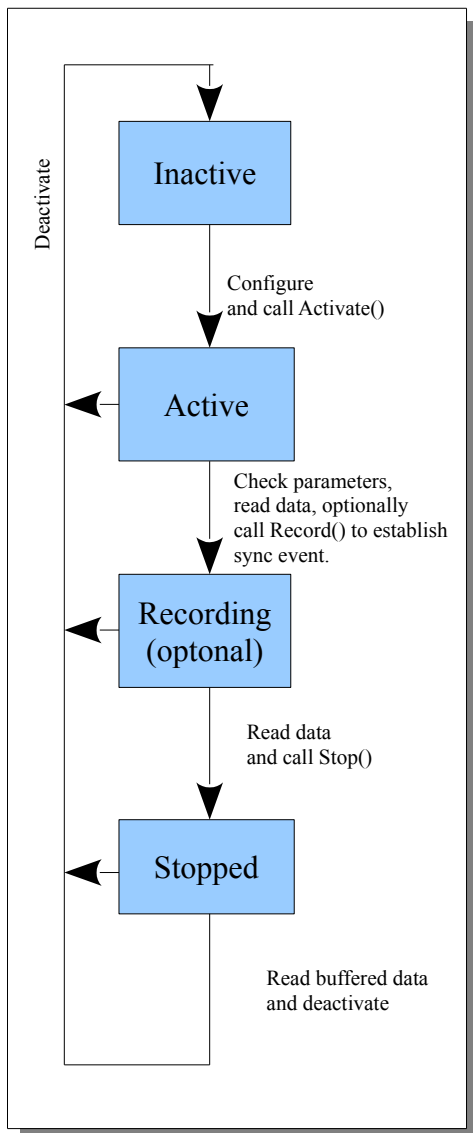
2 Operating cycle

Together with the library, we supply samples in C++ and Visual Basic.NET, which illustrate the acquisition operating cycle. Here we give a brief overview of the necessary steps and go into the details later.

All the interfaces and the objects are defined in the name space of the type library Easy2AcquireCom (e.g. Easy2AcquireCom.DeviceManager), but for sake of simplicity, we omit this name space later in this documentation (e.g. we simply write DeviceManager).

1. Create an instance of the object DeviceManager and initialize it (IDeviceManager.Initialize).
2. Execute the method IDeviceManager.Setup to configure the devices installed at your location. This step is optional if it had been done earlier.
3. Obtain an instance of the currently-selected device (virtual multi-device) by calling the method IDeviceManager.GetCurrentDevice.
4. Retrieve the necessary components of the device, enable them (IComponent.Enable) and configure their acquisition parameters.
5. Activate the device by IDevice.Activate, and review the granted acquisition parameters (e.g. granted sampling frequency may differ from the requested one).
6. Start the device by IDevice.Start.
7. Read the acquired data by periodically calling IDevice.Transfer and picking the arrived data from the library-allocated buffers.
8. Stop the device by IDevice.Stop.
9. Read the pending data (optional step) until IDevice.Transfer reports that no more data are expected.
10. Deactivate the device by IDevice.Deactivate. If you want to continue acquisition, disable the components which are not needed any more (IComponent.Disable) and proceed to the step 4.
11. Release the device and the device manager.

Here is the device state diagram:



3 Handling errors

Errors are handled using the last-error concept. All COM methods return HRESULT which is either S_OK or E_FAIL. In the latter case, the method IDeviceManager.GetLastError can be used to get a human-readable description of the last error.

To prevent the typical design errors caused by calling operations at wrong times, we took special efforts to check if the called method makes any sense in the current state, and if not, we return an error telling that the method is unavailable in the current state.

If IDevice.Activate is unable to start the system, the device returns back to the inactive state, so it is possible to examine the error description and try again.

IDevice.Deactivate is available in all states.

4 Setup

The information about the installed devices is managed by the device manager of the library (the object `DeviceManager`). The setup of this device shows a list of profiles, and each profile includes multiple devices, completely configured and ready to use. You can quickly switch from profile to profile if you move with your laptop from one lab to another.

The devices can be added, removed, or reordered within a profile, but once a device is inserted into a profile, its hardware features and options must be fully known. Here is the reason why: the profile can include multiple devices, and the entire configuration can be quite complex. From the other hand, there can be applications that are using only a subset of the hardware, and in such a case, the unnecessary devices may even be off, but the client application must still be able to review the entire set of the installed hardware.

The currently-selected profile (a multi-device supporting the interface `IDevice`) can be retrieved by `DeviceManager.GetCurrentDevice`.

The entire configuration of the library is stored in a dedicated directory, which must be passed as an argument to `IDeviceManager.Initialize`.

5 Components

5.1 Component types

Each input line (acquisition channel) is a component (an object with an interface derived from `IComponent`). No matter how many actual devices are configured in the profile, all their components are available in one list available in `IDevice`. Currently, the library defines these type of components:

- `IDigitalInput` – a digital input line sampled with a fixed frequency, each data quant encoded as one byte, carrying the value 0 (off) or 1 (on).
- `IAnalogInput` – an 1D input line sampled with a fixed frequency, each data quant encoded in 64-bit floating point. Usually, the data comes in Volts or other measurement units, which can be obtained in `IAnalogInput.GetUnits`
- `IMotionInput` – a 3D input line sampled with a fixed frequency, each data quant encoded as 3x64-bit floating point value. Used for 3D coordinates and 3D orientation. Coordinates are expressed in mm, orientation is expressed as the vector part of the orientation quaternion.
- `IMatrixInput` – a surface distribution input line sampled with a fixed frequency, each data quant encoded as WxH 8-bit unsigned integers, its physical units and conversion coefficient can be obtained by `IMatrixInput.GetUnits` and `IMatrixInput.GetScale` correspondingly.

Other component types (even non-input component types) can be later added.

5.2 Id, tags, and enumeration

Each component has a persistent identifier retrieved by `IComponent.GetId`, the component can be obtained from the device by this identifier using `IDevice.GetComponentById`.

Each component has a list of the associated tags (`IComponent.GetTags`) separate by semicolon (“;”) which contain information about the component types and topology. The tags beginning with...

- device.** denote a logical device, the component is associated with.
- type.** mark the interfaces supported by this component (e.g. components with the tag “type.input.analog” support the interface `IAnalogInput`).
- line.** refer to the name of the channel.

The tags are always presented in the technical format (C-naming convention). Their localized user-friendly names can be retrieved by `IDevice.GetTagName`.

In some cases, the component can include several tags with the same prefix. E.g. if a component is associated with two logical devices, its id will have two tags starting with “device.”.

The enumeration of the components is done with the functions `IDevice.GetComponentCount` and `IDevice.GetComponent` (by a zero-based index). But tags can be conveniently used to restricted enumeration to only the components with desired tags. To do it, call `IDevice.SetComponentFilterTags` (it accepts tags separated by a semicolon) prior to calling `IDevice.GetComponentCount` and `IDevice.GetComponent`. So, to enumerate only the analog input devices, call `SetComponentFilterTags(“type.input.analog”)`. To make the filter stronger, call `IDevice.AddComponentFilterTags`. To clear the filter and enumerate all components again, call `IDevice.SetComponentFilterTags`, passing the empty string as the argument.

5.3 Selecting components for measurement

If the component is enabled (`IComponent.Enable`), the next activation (`IDevice.Activate`) will include it into the acquisition process. If the device is not needed any more, it can be excluded from the next activation by `IComponent.Disable`. Please note that Enable/Disable operations are counted, so if a component was enabled 10 times, to disable it, `IComponent.Disable` must be called also 10 times.

6 Data transmission

6.1 Where to get the acquired data from

Data from the input components are transmitted through the component-allocated buffers. The client application calls `IDevice.Transfer`, which fills the transmission state mask passed as an argument. If ANDed with 2 (`TransferDataReady`) gives non-zero, the data has arrived to at least one of the components.

Each input component has a method `GetQuants`, used to copy the data from the component buffers to the buffers of the client application. The component buffer will keep only the data arrived during the last transfer, and the number of arrived quants can be retrieved by `GetQuantCount`.

6.2 Transmission and timing

Each input component has a set of methods to control acquisition timing and sample status. The client application can call the method `SetDesiredFrequency` to set the sampling frequency the client application would like to sample with. After activation, the client application should call `GetFrequency` to retrieve the actual frequency granted by the acquisition hardware.

To simplify creation of the client buffers, each input component has a method `CreateCompatibleBuffer`, which is available in the active state (after successful `IDevice.Activate`), and will fill the `VARIANT` passed as an argument with a compatible `SAFEARRAY`, which will be big enough to hold the maximum data portion for one transmission (`GetMaxQuantCount` – available after activation).

The method `GetStartTimeHW` (available after `IDevice.Deactivate`) returns the time in seconds (64-bit floating-point) when the sampling of the component started relatively to the global hardware sync event. If hardware sync is not available for the given hardware, it returns -NAN.

The method `GetStartTimeSW` (available after `IDevice.Deactivate`) returns the time in seconds (64-bit floating-point) when the sampling of the component started relatively to the global software sync event (client application generates it by calling `IDevice.Record`, available after `IDevice.Start` and before `IDevice.Stop`).

So, if your devices support hardware synchronization, you can align the data streams after recording by using the time information returned by `GetStartTimeHW`. If some of your device do not support hardware synchronization, you can still roughly align them using the information return by `GetStartTimeSW`.

6.3 Data transfer operation

The data are accumulated in the hardware buffers of the device and in the buffers of the system drivers. The capacity of the accumulation buffers differs for different hardware, but in any case it must be big enough to overcome the typical drawing/recording delays of Windows applications. When the client application gets CPU time, it should be able to read-out all the possible accumulated data.

After each `IDevice.Transfer`: the client application can query all used input component objects how much data has arrived (`GetQuantCount`), and then pick the data from the components using `GetQuants`.

6.4 Reading out the remaining data after stop

After calling `IDevice.Stop`, some data may still remain in the hardware buffers, and the client application can still `IDevice.Transfer` until it stops reporting `TransferMoreExpected`. The data are transferred as usually. However, all this processing is optional, and you can just call `IDevice.Deactivate`, then it is not needed to read-out the buffers.

6.5 Post-acquisition data retrieval

Some device components support functionality of retrieving data in a post-acquisition step. This involves the following steps:

1. Prior to the first activation call `RecoveryEnable` for the components you would like to be included into the post-acquisition operation. If the operation is supported it is enabled and the function returns `TRUE`. Otherwise the function returns `FALSE`.
2. Enable the components and perform the normal acquisition operation.
3. After deactivating the device, read the session identifier using `IDevice::GetSession`.
4. Call `RecoveryStart` on the component, passing the session identifier as the argument.
5. For each block call `RecoveryBlockAsk`, passing the number of the first quant and the quant count.
6. For each block call `RecoveryBlockGet`, which fills the passed buffer with the retrieved data.
7. Repeat the steps 4-6 for all the desired components.

7 Redistribution

When redistributing, it is necessary to register the library `Easy2.AcquireCom.dll`. Please remember that registration needs administrator rights, which must be explicitly enabled in Windows Vista and Windows 7. The DLLs used by `Easy2.AcquireCom.dll` must be in the DLL search path.

8 Interface Reference

All classes and interfaces reside in the namespace of the library (e.g. Easy2AcquireCom.DeviceManager). Besides the native interfaces, all the classes support automation through IDispatch.

All the methods return HRESULT with either S_OK or E_FAIL. The result (if needed) is returned into the last argument. Normally, when a COM library is imported, raw COM methods are automatically wrapped into a method which converts the resulting argument to the return-syntax and which throws an exception if the HRESULT indicates a failure. In this reference additionally to the arguments and the returning value of the methods we include the raw syntax as it is defined in the IDL file.

If a method returns E_FAIL (the wrapper would throw an exception), the error text can be obtained with DeviceManager::GetLastErrorText.

8.1 IDeviceManager

This is the root interface of the library. It is obtained by instantiating a co-class DeviceManager and has the following methods:

8.1.1 Initialize (ConfigDir String)

Raw syntax: HRESULT Initialize([in] BSTR ConfigDir)

Initializes the Easy2.AcquireCom library. The argument ConfigDir must specify the directory where the configuration files are stored. If ConfigDir is any empty string, the configuration files are expected in the same directory where Easy2.AcquireCom.dll is located.

The configuration is stored in two parts:

1. The file Easy2.Acquire.edition and the directory Easy2.Acquire.edition# contain installation-specific data which never changes after installation.
2. The file Easy2.Acquire.data and the directory Easy2.Acquire.data# carry information about the configuration of the user hardware and can be altered by DeviceManager.Setup. Their location can be redirected by the entry data_path in the XML file Easy2.Acquire.edition.

The second call to this function is ignored.

8.1.2 Setup (hWndParent Window), returns Bool

Raw syntax: HRESULT Setup(OLE_HANDLE hWndParent, [out, retval] BOOL* ok)

Executes the configuration dialog. Returns TRUE if the user presses OK and to FALSE if setup was canceled. The argument hWndParent is the handle of the parent window which will be disabled until the modal operation is over.

8.1.3 GetCurrentDevice (), returns IDevice

Raw syntax: HRESULT GetCurrentDevice([out, retval] IDevice** ppIDevice)

Returns IDevice interface of the currently-configured hardware. In multi-device configurations, it will be a wrapper which presents all selected devices as a single device.

8.1.4 GetLastErrorText (), returns String

Raw syntax: HRESULT GetLastErrorText([out, retval] BSTR* ErrorText)

Returns an error message set by the last function of Easy2.AcquireCom which returned a failure code.

8.1.5 ClearLastErrorText ()

Raw syntax: HRESULT ClearLastErrorText()

Clears the last error message.

8.2 IDevice

Performs component enumeration and manages state transition synchronously for all the involved hardware.

All components (input channels) are presented in one plain list even in multi-device configurations. However, each component has a unique string identifier which consists of several tags, which describe the component type, the name or the input channel how it is presented to the user and the name of the box it belongs to.

8.2.1 GetComponentById (Id String), returns IComponent

Raw syntax: HRESULT GetComponentById([in] BSTR Id, [out, retval] IComponent** Component)

Returns a component by its string identifier. This method ignores any enumeration filter.

8.2.2 GetComponentCount (), returns Integer

Raw syntax: HRESULT GetComponentCount([out, retval] LONG* Count)

Returns the number of components matching the enumeration filter.

8.2.3 GetComponent (Index Integer), returns IComponent

Raw syntax: HRESULT GetComponent([in] LONG Index, [out, retval] IComponent** Component)

Returns a component by index from the list of the components matching the enumeration filter.

8.2.4 ClearComponentFilter ()

Raw syntax: HRESULT ClearComponentFilter()

Clears the enumeration filter, so the methods GetComponentCount and GetComponent enumerate all the components. This is the default state when the device obtained by DeviceManager.GetCurrentDevice.

8.2.5 AddComponentFilterTags (Tags String)

Raw syntax: HRESULT AddComponentFilterTags([in] BSTR Tags)

Adds passed tags as the enumeration filter for the methods GetComponentCount and GetComponent. Tags are given in a list separated by a semi-colon (;). Each tag consists of dot (.)-separated tokens with detalization increasing to the right.

Examples:

1. “type.input.analog” limits enumeration to all analog input channels.
2. “type.input.analog.voltage” limit enumeration to all analog input channels which measure raw voltage.
3. “type.input.analog.angle” limit enumeration to all analog channels which measure angle.
4. “type.input.analog;device.vendor” limits enumeration to all analog input channels of a device of a specific vendor.

8.2.6 SetComponentFilterTags (Tags String)

Raw syntax: HRESULT SetComponentFilterTags([in] BSTR Tags)

Just a combination of ClearComponentFilter and AddComponentFilterTags(Tags)

8.2.7 GetTagsByPrefix (Prefix String), returns Array of String

Raw syntax: HRESULT GetTagsByPrefix([in] BSTR Prefix, [out, retval] SAFEARRAY(BSTR)* Tags)

Returns an array of all component tags which start with the passed prefix.

Examples:

1. GetTagsByPrefix("device") returns array of tags starting with "device."
2. GetTagsByPrefix("type.input") returns array with all input type tags registered in the current hardware configuration.

8.2.8 GetTagName (Tag String), returns String

Raw syntax: HRESULT GetTagName([in] BSTR Tag, [out, retval] BSTR* Name)

Returns the user name of the passed tag.

8.2.9 Activate ()

Raw syntax: HRESULT Activate()

Transitions from inactive to active state.

It passes the selected configuration to the hardware and waits until all the devices with enabled components are running. After activation, the client application must query the components it enabled for the granted parameters (e.g. actual sampling frequency), which may be different from the desired ones.

If activation fails, the system rolls back to inactive state.

8.2.10 GetSession (), returns String

Raw syntax: HRESULT GetSession([out, retval] BSTR* Session)

Returns the session ID, which is a unique string generated at each activation. This ID is passed to the method RecoveryStart of the components supporting post-acquisition data retrieval.

8.2.11 Transfer (), returns mask

Raw syntax: HRESULT Transfer([out, retval] LONG* State)

Available in state: started.

Performs an atomic data transfer between the system buffers and the buffers which can be accessed by the client application.

Returns a mask as a combination of the values from Easy2AcquireCom.Transfer:

- TransferDataReady = 1 indicates that some data was transferred during this call.
- TransferMoreExpected = 2 indicates that some data are still pending in the hardware or system buffers.

8.2.12 Record ()

Raw syntax: HRESULT Record()

Transitions from started to recording.

This function generates hardware and software sync events. It must be called when recording starts. All running inputs set their sync_sw to the time when Record was called minus the time when sampling started. In other words,

sync_sw = -preview_duration.

Devices may add additional correction to improve software synchronization.

This method also causes the sync output pulse generated for the hardware which supports it. This pulse received by the inputs which can sense it. These inputs calculate set their sync_hw:

$\text{sync_hw} = \text{time_of_first_quant} - \text{time_of_hw_event}$

Both, `sync_sw` and `sync_hw` times are usually negative because the first quant is sampled before the hardware and software sync events.

8.2.13 Stop ()

Raw syntax: `HRESULT Stop()`

Transitions from started or recording to stopped.

Stops the started devices. The client application may still call Transfer to read out data from the hardware or system buffers.

8.2.14 Deactivate ()

Raw syntax: `HRESULT Deactivate()`

Transitions from any state to inactive state.

Deactivates the activated components. If some devices need long time for secondary activation, they can remain in a warm state until timeout happens or the `IDevice` is released.

8.3 IComponent

This is a base interface for all components.

8.3.1 GetId (), returns String

Raw syntax: HRESULT GetId([out, retval] BSTR* Id)

Returns the ID of the component.

8.3.2 GetTags (), returns Array of String

Raw syntax: HRESULT GetTags([out, retval] SAFEARRAY(BSTR)* Tags)

Splits the entire component ID string to separate tags and returns them as an array of strings.

8.4 IAnalogInput

This is a component interface which provides access to a single analog input channel. The data are sampled in 64-bit floating-point values. Analog inputs have type tags beginning from “type.input.analog”.

This interface implements the methods of IComponent plus the following methods:

8.4.1 Enable ()

Raw syntax: HRESULT Enable()

Available in state: inactive

Includes the component into the next activation.

8.4.2 Disable ()

Raw syntax: HRESULT Disable()

Available in state: any

Excludes the component from the next activation.

8.4.3 Enabled (), returns Integer

Raw syntax: HRESULT Enabled([out, retval] LONG* Count)

Available in state: any

Returns the number of times Enable was called for this component.

8.4.4 GetMaxQuantCount (), returns Integer

Raw syntax: HRESULT GetMaxQuantCount([out, retval] LONG* MaxQuantCount)

Available in state: active

Returns the maximal transmission size in quants.

8.4.5 SetDesiredFrequency (DesiredFrequency Float64)

Raw syntax: HRESULT SetDesiredFrequency([in] DOUBLE DesiredFrequency)

Available in state: inactive

Sets the desired sampling frequency for this component. Please note that the actual sampling frequency may differ. It should be retrieved after activation using the method GetFrequency.

It is optional to call this function, the device should run with a proper default value.

8.4.6 GetDesiredFrequency (), returns Float64

Raw syntax: HRESULT GetDesiredFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: any

Returns the last value passed by SetDesiredFrequency or a reasonable default value if SetDesiredFrequency was never called.

8.4.7 GetFrequency (), returns Float64

Raw syntax: HRESULT GetFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: active

Returns the actual sampling frequency. After activation, the client application must call this function to obtain the actual granted value.

8.4.8 GetUnits (), returns String

Raw syntax: HRESULT GetUnits([out, retval] BSTR* Units)

Available in state: any

Returns the measurement units of this analog input (e.g. “V” for voltage input lines).

8.4.9 CreateCompatibleBuffer (Buffer Variant)

Raw syntax: HRESULT CreateCompatibleBuffer(VARIANT* buffer)

Available in state: active

This is a helper function to for simplifying creation of the data transmission buffer on the client side. It fills the given argument with a buffer suitable for GetQuants: SAFEARRAY of Float64 with indexes from 0 to GetMaxQuants()-1.

8.4.10 GetQuantCount (), returns Integer

Raw syntax: HRESULT GetQuantCount([out, retval] LONG* QuantCount)

Available in state: started

Returns the number of quants arrived during the last IDevice.Transfer. They can be retrieved by GetQuants.

8.4.11 GetQuants (SrcIndex Integer, Count Integer, Buffer Variant, DstIndex Integer)

Raw syntax: HRESULT GetQuants(LONG SrcIndex, LONG Count, VARIANT* Buffer, LONG DstIndex)

Available in state: started

Copies arrived data to the application buffer. The buffer can be allocated with CreateCompatibleBuffer or defined as SAFEARRAY of Float64.

The data during IDevice.Transfer always arrives at the beginning of the buffer, and the arrived quant count cannot exceed the value returned by GetMaxQuantCount.

Usually, the client application retrieves the entire arrived data portion by calling

Input.GetQuants(0, Input.GetQuantCount(), Buffer, 0)

But it is also possible to read a part of it by specifying the proper SrcIndex, Count, and DstIndex.

8.4.12 GetStartTimeHW (), returns Float64

Raw syntax: HRESULT GetStartTimeHW([out, retval] DOUBLE* StartTimeHW)

Available in state: inactive

Returns

time_of_first_quant – time_of_hw_sync_event

Usually, this value is negative, because HW sync event is received after sampling was started.

If the input does not support sync in feature or the hardware sync even was not received, this method returns NaN.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.4.13 GetStartTimeSW (), returns Float64

Raw syntax: HRESULT GetStartTimeSW([out, retval] DOUBLE* StartTimeSW)

Available in state: inactive

Returns

$\text{time_of_first_quant} - \text{time_of_sw_sync_event}$

sw_sync_event takes place when IDevice.Record is called.

Usually, this value is negative, because Record is called after sampling was started. Implementers may add additional fixup.

If IDevice.Record was never called, this value is equal to an hardware-specific constant value.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.4.14 RecoveryEnable(), returns BOOL

Raw syntax: HRESULT RecoveryEnable([out, retval] BOOL* Supported)

Available in state: before the first activation

Enables post-acquisition data retrieval if supported, and returns TRUE. The values lost during data transmission are filled with -NAN.

If the post-acquisition data retrieval is not supported, returns FALSE.

8.4.15 RecoveryStart(String Session)

Raw syntax: HRESULT RecoveryEnable([in] BSTR Session)

Available in state: inactive, after RecoveryEnable was called.

Starts post-acquisition data retrieval from component. The hardware may keep data from multiple sessions, so please pass as the argument the session ID obtained from IDevice::GetSession after activation.

The retrieval queue is cleared from the previously requested blocks.

8.4.16 RecoveryBlockAsk(Index Integer64, Count Integer)

Raw syntax: HRESULT RecoveryBlockAsk(__int64 Index, LONG Count)

Available in state: inactive, after RecoveryStart was called.

Requests a data block to be retrieved from the device as a post-acquisition data retrieval operations. RecoveryStart must be executed prior to calling this function.

8.4.17 RecoveryBlockGet(Buffer Variant, DstIndex Integer)

Raw syntax: HRESULT RecoveryBlockGet(VARIANT* Buffer, LONG DstIndex)

Available in state: inactive, after RecoveryBlockGet was called.

Reads one data block previously-requested with RecoveryBlockAsk and removes it from the queue.

DstIndex is the index, starting from which the data are placed into in the buffer.

8.5 IDigitalInput

This is a component interface which provides access to a single digital input channel. The data are sampled in byte quants each taking a value 0 or 1. Digital inputs have type tags “type.input.digital”.

This interface implements the methods of IComponent plus the following methods:

8.5.1 Enable ()

Raw syntax: HRESULT Enable()

Available in state: inactive

Includes the component into the next activation.

8.5.2 Disable ()

Raw syntax: HRESULT Disable()

Available in state: any

Excludes the component from the next activation.

8.5.3 Enabled (), returns Integer

Raw syntax: HRESULT Enabled([out, retval] LONG* Count)

Available in state: any

Returns the number of times Enable was called for this component.

8.5.4 GetMaxQuantCount (), returns Integer

Raw syntax: HRESULT GetMaxQuantCount([out, retval] LONG* MaxQuantCount)

Available in state: active

Returns the maximal transmission size in quants.

8.5.5 SetDesiredFrequency (DesiredFrequency Float64)

Raw syntax: HRESULT SetDesiredFrequency([in] DOUBLE DesiredFrequency)

Available in state: inactive

Sets the desired sampling frequency for this component. Please note that the actual sampling frequency may differ. It should be retrieved after activation using the method GetFrequency.

It is optional to call this function, the device should run with a proper default value.

8.5.6 GetDesiredFrequency (), returns Float64

Raw syntax: HRESULT GetDesiredFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: any

Returns the last value passed by SetDesiredFrequency or a reasonable default value if SetDesiredFrequency was never called.

8.5.7 GetFrequency (), returns Float64

Raw syntax: HRESULT GetFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: active

Returns the actual sampling frequency. After activation, the client application must call this function to obtain the actual granted value.

8.5.8 CreateCompatibleBuffer (Buffer Variant)

Raw syntax: HRESULT CreateCompatibleBuffer(VARIANT* buffer)

Available in state: active

This is a helper function to for simplifying creation of the data transmission buffer on the client side. It fills the given argument with a buffer suitable for GetQuants: SAFEARRAY of BYTE with indexes from 0 to GetMaxQuants()-1.

8.5.9 GetQuantCount (), returns Integer

Raw syntax: HRESULT GetQuantCount([out, retval] LONG* QuantCount)

Available in state: started

Returns the number of quants arrived during the last IDevice.Transfer. They can be retrieved by GetQuants.

8.5.10 GetQuants (SrcIndex Integer, Count Integer, Buffer Variant, DstIndex Integer)

Raw syntax: HRESULT GetQuants(LONG SrcIndex, LONG Count, VARIANT* Buffer, LONG DstIndex)

Available in state: started

Copies arrived data to the application buffer. The buffer can be allocated with CreateCompatibleBuffer or defined as SAFEARRAY of BYTE.

The data during IDevice.Transfer always arrives at the beginning of the buffer, and the arrived quant count cannot exceed the value returned by GetMaxQuantCount.

Usually, the client application retrieves the entire arrived data portion by calling

Input.GetQuants(0, Input.GetQuantCount(), Buffer, 0)

But it is also possible to read a part of it by specifying the proper SrcIndex, Count, and DstIndex.

8.5.11 GetStartTimeHW (), returns Float64

Raw syntax: HRESULT GetStartTimeHW([out, retval] DOUBLE* StartTimeHW)

Available in state: inactive

Returns

time_of_first_quant – time_of_hw_sync_event

Usually, this value is negative, because HW sync event is received after sampling was started.

If the input does not support sync in feature or the hardware sync even was not received, this method returns NaN.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.5.12 GetStartTimeSW (), returns Float64

Raw syntax: HRESULT GetStartTimeSW([out, retval] DOUBLE* StartTimeSW)

Available in state: inactive

Returns

time_of_first_quant – time_of_sw_sync_event

sw_sync_event takes place when IDevice.Record is called.

Usually, this value is negative, because Record is called after sampling was started. Implementers

may add additional fixup.

If `IDevice.Record` was never called, this value is equal to an hardware-specific constant value.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.6 IMotionInput

This is a component interface which provides access to a single input channel with 3D position or orientation. Data are sampled in 3 Float64 values. The position channels have type tag “type.input.motion.pos” and stores x,y,z coordinates in mm. The orientation channels have the type tag “type.input.motion.rot” and stores the vector component for orientation quaternion.

This interface implements the methods of IComponent plus the following methods:

8.6.1 Enable ()

Raw syntax: HRESULT Enable()

Available in state: inactive

Includes the component into the next activation.

8.6.2 Disable ()

Raw syntax: HRESULT Disable()

Available in state: any

Excludes the component from the next activation.

8.6.3 Enabled (), returns Integer

Raw syntax: HRESULT Enabled([out, retval] LONG* Count)

Available in state: any

Returns the number of times Enable was called for this component.

8.6.4 GetMaxQuantCount (), returns Integer

Raw syntax: HRESULT GetMaxQuantCount([out, retval] LONG* MaxQuantCount)

Available in state: active

Returns the maximal transmission size in quants.

8.6.5 SetDesiredFrequency (DesiredFrequency Float64)

Raw syntax: HRESULT SetDesiredFrequency([in] DOUBLE DesiredFrequency)

Available in state: inactive

Sets the desired sampling frequency for this component. Please note that the actual sampling frequency may differ. It should be retrieved after activation using the method GetFrequency.

It is optional to call this function, the device should run with a proper default value.

8.6.6 GetDesiredFrequency (), returns Float64

Raw syntax: HRESULT GetDesiredFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: any

Returns the last value passed by SetDesiredFrequency or a reasonable default value if SetDesiredFrequency was never called.

8.6.7 GetFrequency (), returns Float64

Raw syntax: HRESULT GetFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: active

Returns the actual sampling frequency. After activation, the client application must call this function to obtain the actual granted value.

8.6.8 CreateCompatibleBuffer (Buffer Variant)

Raw syntax: HRESULT CreateCompatibleBuffer(VARIANT* buffer)

Available in state: active

This is a helper function to for simplifying creation of the data transmission buffer on the client side. It fills the given argument with a buffer suitable for GetQuants: SAFEARRAY Float64[0..2][0..GetMaxQuants()-1].

8.6.9 GetQuantCount (), returns Integer

Raw syntax: HRESULT GetQuantCount([out, retval] LONG* QuantCount)

Available in state: started

Returns the number of quants arrived during the last IDevice.Transfer. They can be retrieved by GetQuants.

8.6.10 GetQuants (SrcIndex Integer, Count Integer, Buffer Variant, DstIndex Integer)

Raw syntax: HRESULT GetQuants(LONG SrcIndex, LONG Count, VARIANT* Buffer, LONG DstIndex)

Available in state: started

Copies arrived data to the application buffer. The buffer can be allocated with CreateCompatibleBuffer or defined as SAFEARRAY of Float64[0..2][0..Count-1].

The data during IDevice.Transfer always arrives at the beginning of the buffer, and the arrived quant count cannot exceed the value returned by GetMaxQuantCount.

Usually, the client application retrieves the entire arrived data portion by calling

Input.GetQuants(0, Input.GetQuantCount(), Buffer, 0)

But it is also possible to read a part of it by specifying the proper SrcIndex, Count, and DstIndex.

8.6.11 GetStartTimeHW (), returns Float64

Raw syntax: HRESULT GetStartTimeHW([out, retval] DOUBLE* StartTimeHW)

Available in state: inactive

Returns

time_of_first_quant – time_of_hw_sync_event

Usually, this value is negative, because HW sync event is received after sampling was started.

If the input does not support sync in feature or the hardware sync even was not received, this method returns NaN.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.6.12 GetStartTimeSW (), returns Float64

Raw syntax: HRESULT GetStartTimeSW([out, retval] DOUBLE* StartTimeSW)

Available in state: inactive

Returns

time_of_first_quant – time_of_sw_sync_event

sw_sync_event takes place when IDevice.Record is called.

Usually, this value is negative, because Record is called after sampling was started. Implementers may add additional fixup.

If IDevice.Record was never called, this value is equal to an hardware-specific constant value.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.6.13 RecoveryEnable(), returns BOOL

Raw syntax: HRESULT RecoveryEnable([out, retval] BOOL* Supported)

Available in state: before the first activation

Enables post-acquisition data retrieval if supported, and returns TRUE. The values lost during data transmission are filled with -NAN.

If the post-acquisition data retrieval is not supported, returns FALSE.

8.6.14 RecoveryStart(String Session)

Raw syntax: HRESULT RecoveryEnable([in] BSTR Session)

Available in state: inactive, after RecoveryEnable was called.

Starts post-acquisition data retrieval from component. The hardware may keep data from multiple sessions, so please pass as the argument the session ID obtained from IDevice::GetSession after activation.

The retrieval queue is cleared from the previously requested blocks.

8.6.15 RecoveryBlockAsk(Index Integer64, Count Integer)

Raw syntax: HRESULT RecoveryBlockAsk(__int64 Index, LONG Count)

Available in state: inactive, after RecoveryStart was called.

Requests a data block to be retrieved from the device as a post-acquisition data retrieval operations. RecoveryStart must be executed prior to calling this function.

8.6.16 RecoveryBlockGet(Buffer Variant, DstIndex Integer)

Raw syntax: HRESULT RecoveryBlockGet(VARIANT* Buffer, LONG DstIndex)

Available in state: inactive, after RecoveryBlockGet was called.

Reads one data block previously-requested with RecoveryBlockAsk and removes it from the queue.

DstIndex is the index, starting from which the data are placed into in the buffer.

8.7 IMatrixInput

This is a component interface which provides access to a single input channel with distribution matrix. Each quant is a 2D array of 8-bit unsigned integer cells.

This interface implements the methods of IComponent plus the following methods:

8.7.1 Enable ()

Raw syntax: HRESULT Enable()

Available in state: inactive

Includes the component into the next activation.

8.7.2 Disable ()

Raw syntax: HRESULT Disable()

Available in state: any

Excludes the component from the next activation.

8.7.3 Enabled (), returns Integer

Raw syntax: HRESULT Enabled([out, retval] LONG* Count)

Available in state: any

Returns the number of times Enable was called for this component.

8.7.4 GetSizeX (), returns Integer

Raw syntax: HRESULT GetSizeX([out, retval] LONG* SizeX)

Available in state: any

Returns number of cells in X direction.

8.7.5 GetSizeY (), returns Integer

Raw syntax: HRESULT GetSizeY([out, retval] LONG* SizeY)

Available in state: any

Returns number of cells in Y direction.

8.7.6 GetResX (), returns Real64

Raw syntax: HRESULT GetResX([out, retval] DOUBLE* ResX)

Available in state: any

Returns the physical width of a cell in mm.

8.7.7 GetResY (), returns Real64

Raw syntax: HRESULT GetResY([out, retval] DOUBLE* ResY)

Available in state: any

Returns the physical height of a cell in mm.

8.7.8 GetRes (), returns Real64

Raw syntax: HRESULT GetRes([out, retval] DOUBLE* ResY)

Available in state: any

Returns the scale which must be multiplied to the integer cell value to get physical cell value in N/cm².

8.7.9 GetMaxQuantCount (), returns Integer

Raw syntax: HRESULT GetMaxQuantCount([out, retval] LONG* MaxQuantCount)

Available in state: active

Returns the maximal transmission size in quants.

8.7.10 SetDesiredFrequency (DesiredFrequency Float64)

Raw syntax: HRESULT SetDesiredFrequency([in] DOUBLE DesiredFrequency)

Available in state: inactive

Sets the desired sampling frequency for this component. Please note that the actual sampling frequency may differ. It should be retrieved after activation using the method GetFrequency.

It is optional to call this function, the device should run with a proper default value.

8.7.11 GetDesiredFrequency (), returns Float64

Raw syntax: HRESULT GetDesiredFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: any

Returns the last value passed by SetDesiredFrequency or a reasonable default value if SetDesiredFrequency was never called.

8.7.12 GetFrequency (), returns Float64

Raw syntax: HRESULT GetFrequency([out, retval] DOUBLE* DesiredFrequency)

Available in state: active

Returns the actual sampling frequency. After activation, the client application must call this function to obtain the actual granted value.

8.7.13 CreateCompatibleBuffer (Buffer Variant)

Raw syntax: HRESULT CreateCompatibleBuffer(VARIANT* buffer)

Available in state: active

This is a helper function to for simplifying creation of the data transmission buffer on the client side. It fills the given argument with a buffer suitable for GetQuants: SAFEARRAY Float64[0..2][0..GetMaxQuants()-1].

8.7.14 GetQuantCount (), returns Integer

Raw syntax: HRESULT GetQuantCount([out, retval] LONG* QuantCount)

Available in state: started

Returns the number of quants arrived during the last IDevice.Transfer. They can be retrieved by GetQuants.

8.7.15 GetQuant (SrcIndex Integer, Buffer Variant)

Raw syntax: HRESULT GetQuant(LONG SrcIndex, VARIANT* Buffer)

Available in state: started

Copies a single quant from the arrived data to the application buffer. The buffer can be allocated with CreateCompatibleBuffer or defined as SAFEARRAY of Float64[0..GetSizeX()-1][0..GetSizeY()-1]

-1].

SrcIndex is the index of the quant in the arrived buffer. At each IDevice.Transfer the data arrives from the beginning of the buffer.

8.7.16 GetStartTimeHW (), returns Float64

Raw syntax: HRESULT GetStartTimeHW([out, retval] DOUBLE* StartTimeHW)

Available in state: inactive

Returns

$\text{time_of_first_quant} - \text{time_of_hw_sync_event}$

Usually, this value is negative, because HW sync event is received after sampling was started.

If the input does not support sync in feature or the hardware sync even was not received, this method returns NaN.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

8.7.17 GetStartTimeSW (), returns Float64

Raw syntax: HRESULT GetStartTimeSW([out, retval] DOUBLE* StartTimeSW)

Available in state: inactive

Returns

$\text{time_of_first_quant} - \text{time_of_sw_sync_event}$

sw_sync_event takes place when IDevice.Record is called.

Usually, this value is negative, because Record is called after sampling was started. Implementers may add additional fixup.

If IDevice.Record was never called, this value is equal to an hardware-specific constant value.

Please note that this method is available only after deactivation because some devices need to analyze transmission statistics to calculate this value.

9 The list of changes

V1.7.25

- Noraxon DTS: 400G sensor added
- Noraxon DTS: respiration sensor added

v1.7.18

- Noraxon DTS: data recovery support added.

v1.7.11

- Noraxon MyoMotion: acceleration support added.
- Noraxon MyoMotion: clinical sensors support added.
- Noraxon ClinicalDTS: non-HID device support added.

v1.7.8

- Superfluous Start() removed. Now devices are started automatically after Activate().
- Noraxon Sync, Noraxon Mt400, Noraxon MyoMotion compatibility.

v1.6.9

- Noraxon 1400, G2 USB, Belt Receiver USB, Desk Receiver: multiple instances enabled.
- Automatic device detection on first start.

v1.6.8

- GetQuants: DstIndex handling fixed.

v1.6.6

- Sync warning shown only if IDevice.Record was called
- SW Sync is NAN if IDevice.Record was not called after activation
- tag consistency improved:
 - “type.input.motion” renamed to “type.input.motion.pos”
 - “type.input.orientation” renamed to “type.input.motion.rot”

v1.6.5

- Noraxon Belt Receiver support implemented.
- Incorrectly-named IMatrixInput.GetQuants fixed to IMatrixInput.GetQuant
- Noraxon Desk Receiver: sync out control implemented.

v1.6.4

- Noraxon Clinical DTS setup: radio channel drops to a1 when entering the dialog – fixed.

v1.6.0

- Senders now deprecated, removed from samples and documentation. Frequency and transfer

information accessed directly by components.

- Noraxon Clinical DTS setup simplified
- Noraxon Desk receiver supported

v1.5.20

- Hardware setup dialog reworked.
- Improvements in Noraxon Clinical DTS Setup: now both manual and automatic sensor-to-channel mapping supported.
- component type tags have subtype suffixes, e.g.:
 - “type.input.analog.velocity” - the component has interface IAnalogInput and delivers there velocity data.
 - “type.input.analog.voltage” - generic-purpose IAnalogInput which captures voltage.
 - “type.input.matrix.pressure” - pressure distribution matrix captured using IMatrixInput interface.
- General filter tags (see IDevice.SetComponentFilterTags / IDevice.AddComponentFilterTags) will match more specific filter tags. E.g. “type.input.analog” will match “type.input.analog.velocity” and “type.input.analog.voltage”; “type.input” will match all the components which have tag starting from “type.input.”.

v1.5.10

- Implemented devices for Zebris FDM, Noraxon 1400, Noraxon G2. The actual set of drivers differ depending on the distribution.
- New installer written, component tags changed from “type_analog_input” to “type.input.analog”, etc...
- ISender.GetLastStartTime changed to ISender.GetStartTimeHW and GetStartTimeSW. ISender.Record for generating sync event added.
- The arrangement of settings and data files changed to match the security rules of Windows:
 - the binary directory has Easy.Acquire.edition, which contain the list of drivers and location for data and temp files and the directory Easy.Acquire.edition# which has firmware. These files are intended to be accessible only for users with elevated privileges.
 - The file Easy.Acquire.data and the directory Easy.Acquire.data# referenced from Easy.Acquire.edition have all the current configurations.

v1.3

- IDevice.GetTagsByPrefix subject to the component filter
- IDevice.ClearComponentFilter added
- device_manager.easyobj renamed to device_manager.easydata, edition settings moved from settings.ini to in edition.easydata

v1.2

- tags **type_XXX_input** renamed to **type_input_XXX**, so that detecting input lines would be easier (see IDevice.GetTagsByPrefix)
- tags should now be retrieved by IComponent.GetTags

v1.1

- Tag support added

- retrieval attributes added to COM interfaces
- ISender.GetLastStartTime added
- Installer added for registering Easy2.AcquireCom.dll under Windows Vista and Windows 7